A MINI PROJECT ON

Finance Management Model Bot

Submitted for the partial fulfillment of the requirement for the degree of

Bachelors of Business Administration in Artificial Intelligence with Honors

Submitted by –

Ashwin Chhawaniya – 25030422028

Arnav Singh – 25030422022

Akshat Goyal – 25030422014

(Semester – 1)

Submitted to –

DR. DAWA CHYOPHEL LEPCHA

Assistant Professor

Python Programming

SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITITUE

CONSTITUENT OF SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

LAVALE, MULSHI, PUNE-411211, MAHARASHTRA

November 2025

CERTIFICATE

This is to certify that the mini project titled **Finance Management Model Bot** has been carried out by **Ashwin Chhawaniya (25030422028)**, **Arnav Singh (25030422022)**, and **Akshat Goyal (25030422014)** of **Bachelor of Business Administration in Artificial Intelligence with Honors (Semester – 1)**, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Business Administration in Artificial Intelligence with Honors**.

The project has been completed under the supervision and guidance of **Dr. Dawa Chyophel Lepcha, Assistant Professor – Python Programming**, at the **Symbiosis Artificial Intelligence Institute, a Constituent of Symbiosis International (Deemed University), Lavale, Mulshi, Pune – 411211, Maharashtra**, during the academic year **2025**.

Dr. Dawa Chyophel Lepcha
Assistant Professor, Python Programming

Head of the Department
Symbiosis Artificial Intelligence Institute
Constituent of **Symbiosis International (Deemed University)**
Lavale, Mulshi, Pune – 411211, Maharashtra

ACKNOWLEDGMENT

We would like to express our heartfelt gratitude to **Dr. Shruti Patil**, *Director, Symbiosis Artificial Intelligence Institute (SAII)*, for providing us with the opportunity, facilities, and academic environment necessary to successfully complete our mini project titled "**Finance Management Model Bot.**" Her leadership and encouragement have been a constant source of inspiration.

We are profoundly thankful to our project guide, **Dr. Dawa Chyophel Lepcha**, *Assistant Professor – Python Programming*, for his invaluable guidance, support, and insightful feedback throughout the course of this project. His mentorship has played a vital role in shaping our understanding of Python and its practical applications in business and finance.

We also extend our gratitude to all the faculty members of **SAII**, whose teaching and support have strengthened our knowledge foundation. Finally, we thank our parents, classmates, and friends for their unwavering encouragement, cooperation, and motivation, which greatly contributed to the successful completion of this project.

Ashwin Chhawaniya – 25030422028
Arnav Singh – 25030422022
Akshat Goyal – 25030422014

DECLARATION

We hereby declare that the mini project titled **FINANCE MANAGEMENT MODEL BOT** has been carried out by **ASHWIN CHHAWANIYA (25030422028)**, **ARNAV SINGH (25030422022)**, and **AKSHAT GOYAL (25030422014)** of **BACHELOR OF BUSINESS ADMINISTRATION IN ARTIFICIAL INTELLIGENCE WITH HONORS (SEMESTER – 1)**, under the guidance of **DR. DAWA CHYOPHEL LEPCHA, ASSISTANT PROFESSOR – PYTHON PROGRAMMING**, at the **SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE (SAII)**, a constituent of **SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)**, Lavale, Mulshi, Pune – 411211, Maharashtra.

We further declare that this project report is the result of our original work and has not been submitted to any other university or institution for the award of any degree or diploma. All sources of information used in the preparation of this report have been duly acknowledged.

Place: SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITTUTE (SAII), SIT BUILDING, 6TH FLOOR

Date: ____ NOVEMBER 2025

Ashwin Chhawaniya – 25030422028

Arnav Singh – 25030422022

Akshat Goyal – 25030422014

ABSTRACT

The mini project titled **FINANCE MANAGEMENT MODEL BOT** is developed using **PYTHON** as the core programming language and **MYSQL** as the database for data storage and management. The primary objective of this project is to design an intelligent, user-friendly financial assistant that helps users efficiently manage, analyze, and forecast their finances.

The system allows users to record income and expenses, plan monthly budgets, calculate taxes, compute compound interest, and perform real-time currency conversions. It also provides visual insights through graphical representations using **MATPLOTLIB**, thereby enhancing decision-making based on financial trends and savings growth.

The project demonstrates the effective integration of **PYTHON PROGRAMMING**, **DATABASE CONNECTIVITY**, and **DATA VISUALIZATION** techniques to create a functional, scalable, and interactive finance management tool. It serves as a practical implementation of theoretical knowledge and highlights the growing role of automation and artificial intelligence in the field of financial management.

Ashwin Chhawaniya – 25030422028
Arnav Singh – 25030422022
Akshat Goyal – 25030422014

FMM - Code Manual

College Finance Bot

(Finance_Bot_FMM.py)

Full Code Manual

Version: 1.0 **Date:** November 3, 2025 **Document Type:** Technical Code Manual

---

Table of Contents

Chapter	Section	Subsection	Description / Title
	4.2.3	—	print_banner()
	4.3	—	Database Connection & Initialization
	4.3.1	—	db_connect()
	4.3.2	—	create_all_tables()
	4.4	—	User Authentication Functions
	4.4.1	—	register_user()
	4.4.2	—	login_user()
	4.5	—	Core Financial Feature Functions
	4.5.1	—	get_user_financial_summary()
	4.5.2	—	record_entry()
	4.5.3	—	monthly_budget_planner()
	4.5.4	—	bill_splitter()
	4.5.5	—	tax_calculator()
	4.5.6	—	compound_interest()
	4.5.7	—	savings_calculator()
	4.5.8	—	currency_converter()
	4.6	—	Main Application Flow
	4.6.1	—	main_menu()
	4.6.2	—	main()
	4.6.3	—	if **name** == "**main**":
Chapter 5: Execution & Usage Guide	5.1	—	Running the Application
	5.2	—	Execution Walkthrough (Screenshot Placeholders)
	5.2.1	—	Execution 1: Registration
	5.2.2	—	Execution 2: Login & Main Menu
	5.2.3	—	Execution 3: Recording an Expense
	5.2.4	—	Execution 4: Planning a Monthly Budget
	5.2.5	—	Execution 5: Calculating Tax
	5.2.6	—	Execution 6: Projecting Compound Interest (Graph)
	5.2.7	—	Execution 7: Converting Currency (Live Rates)
	5.2.8	—	Execution 8: Logout
Chapter 6: Conclusion	6.1	—	Document Summary

Chapter 1: Introduction

1.1. Purpose of this Manual

This document provides a comprehensive technical overview and code manual for the **College Finance Bot** (`Finance_Bot_FMM.py`). The purpose of this manual is to serve as a complete guide for developers, system administrators, and end-users who wish to understand, install, operate, or modify the application.

It meticulously details every aspect of the project, including:

- The high-level purpose and all key features.
- The required software, libraries, and setup steps.
- A deep dive into the MySQL database schema, explaining all 8 tables and their columns.
- A complete analysis of the Python source code, explaining the logic of every function.
- A guide on how to execute and use the application, with placeholders for visual examples.

1.2. Application Overview

The application is built on a user-centric model. All data is private and tied to a specific user account, which is protected by a username and password. This data is stored persistently in a MySQL database, meaning a user's financial records are saved even after the application is closed.

Once logged in, the user is presented with a menu of 8 core financial tools, ranging from simple income/expense tracking and bill splitting to complex financial modeling like compound interest projection and savings calculation, complete with data visualizations.

1.3. Target Audience

This manual is written for several audiences:

- **Developers:** Developers who wish to extend the application's functionality, fix bugs, or understand its architecture will find the in-depth code and database analysis in Chapters 3 and 4 invaluable.
- **System Administrators:** Individuals responsible for deploying the application on a server or setting up the database will use the installation and configuration guides in Chapter 2.
- **Code Reviewers / Instructors:** This manual serves as complete documentation for a project submission, demonstrating a thorough understanding of the code, database design, and application flow.
- **Power Users:** Curious end-users who want to understand the inner workings of the tools they are using, such as the exact formulas used in the tax or interest calculators.

1.4. Key Features

The application is feature-rich and provides the following 8 core modules:

1. **Monthly Budget Planner:** Allows a user to define a total monthly budget and allocate it by percentage across three custom priorities (e.g., "Rent", "Food", "Tuition") and a "Miscellaneous" category.
2. **Bill Splitter:** A simple utility that takes a total bill amount and a number of people, then calculates the equal share per person.
3. **Tax Calculator:** A basic income tax calculator modeled on a simplified Indian tax slab system. It calculates the total tax due and the effective tax rate.
4. **Compound Interest Calculator:** A powerful projection tool. The user inputs a principal amount, monthly contribution, duration, and interest rate. The application calculates the future value and plots the growth (including variance) using `matplotlib` .
5. **Currency Converter:** A robust tool to convert between 10 different global currencies. It uniquely offers the choice between using **live, real-time exchange rates** (via the `forex-python` API) or **fixed, offline rates** (hard-coded in

the script).

6. **Savings Calculator:** A projection tool that calculates the future balance of a savings plan based on fixed monthly savings and an annual return rate. This is also plotted on a graph.
7. **Record Income/Expense:** The core ledger of the application. Users can record income or expense transactions, which are tagged and timestamped. The application provides an immediate summary of their new financial position.
8. **Secure User Authentication:** A persistent user account system that allows users to register a new account or log in to an existing one, ensuring all financial data is kept private.

1.5. Technology Stack

The application is built using a simple but powerful stack of open-source technologies:

- **Programming Language: Python 3.x**. A high-level, versatile language chosen for its readability and extensive ecosystem of third-party libraries.
- **Database: MySQL**. A robust, open-source relational database management system (RDBMS) used to store all user and financial data persistently.
- **Python Libraries:**
  - `mysql-connector-python` : The official Oracle driver for connecting Python applications to MySQL databases.
  - `matplotlib` : A comprehensive library for creating static, animated, and interactive visualizations in Python. Used for the compound interest and savings graphs.
  - `forex-python` : A simple library for fetching real-time currency exchange rates and performing conversions.
  - `datetime` : A built-in Python module used for timestamping all relevant transactions and records.

Chapter 2: System & Setup Requirements

2.1. Software Requirements

To run this application, the following software must be installed on your system:

- **Python 3:** The application is written in Python 3 (version 3.6 or newer is recommended). You can download it from `python.org` .
- **MySQL Server:** A running MySQL server (version 5.7 or newer; 8.0+ recommended) is required to act as the database. You can use MySQL Community Server, XAMPP, WAMP, or any other MySQL distribution. The server must be accessible from the machine running the script (e.g., `localhost`).

2.2. Python Library Dependencies

The script relies on three external Python libraries that must be installed using `pip` , the Python package installer.

2.2.1. `mysql-connector-python`

- **Purpose:** This is the official Oracle-supported driver that allows the Python script to communicate with the MySQL database server. It is used for all database operations: connecting, creating tables, inserting data (`INSERT`), and querying data (`SELECT`).
- **Installation:**

```
pip install mysql-connector-python
```

2.2.2. `matplotlib`

- **Purpose:** In this application, it is used to generate graphs for the Compound Interest Calculator and the Savings Calculator, providing the user with a clear visual representation of their financial projections.
- **Installation:**

```
pip install matplotlib
```

2.2.3. `forex-python`

- **Purpose:** This library provides a simple interface to fetch current and historical foreign exchange (forex) rates. It is used in the Currency Converter module to provide live, up-to-the-date-conversion rates.
- **Installation:**

```
pip install forex-python
```

2.3. Database Configuration (`DB_CONFIG`)

Before running the application, you **MUST** configure the database connection settings. Open the `Finance_Bot_FMM.py` file in a text editor and locate the `DB_CONFIG` dictionary near the top of the file.

```
DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "javamx",
    "database": "college_finance_db"
}
```

You must edit these values to match your local MySQL server setup:

- `"host"` : This is typically `localhost` or `127.0.0.1` if the database is on the same machine.
- `"user"` : The username for your MySQL server. The default is often `root` .
- `"password"` : The password you set for that MySQL user. **You must change `"javamx"` to your own password.**
- `"database"` : The name of the database the script will use. You can leave this as `"college_finance_db"` . The script will **automatically create this database** if it doesn't exist.

2.4. Installation Guide

Follow these steps to get the application running:

1. **Install MySQL:** Download and install MySQL Community Server (or a package like XAMPP) and ensure the server is running.
2. **Install Python:** Download and install Python 3 from `python.org` .
3. **Install Libraries:** Open a terminal or command prompt and run the three `pip install` commands from section 2.2.
4. **Configure the Script:** Open `Finance_Bot_FMM.py` and edit the `DB_CONFIG` dictionary with your MySQL username and password as described in section 2.3.

5. **Run the Application:** Open your terminal, navigate to the directory where you saved the script, and run:

```
python Finance_Bot_FMM.py
```

On the first run, the script will automatically connect to MySQL, create the `college_finance_db` database, and create all 8 required tables.

Chapter 3: Database Schema

3.1. Database Overview

The application uses a single MySQL database named `college_finance_db`. This database contains 8 tables, each responsible for storing a specific piece of information. The `user_id` column, which is present in almost every table, acts as a **foreign key** linking records to a specific user in the `users` table. This ensures that a user can only access their own financial data.

This chapter details the schema and purpose of all 8 tables.

3.2. Table 1: `users`

- **Purpose:** It stores the user account information, including their unique username and password. All other tables link back to the `id` in this table.
- **SQL Definition:**

```
CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL
) ENGINE=InnoDB
```

- **Column Breakdown:**

Column	Type	Constraints	Purpose
`id`	`INT`	`AUTO_INCREMENT`, `PRIMARY KEY`	A unique number for each user.
`username`	`VARCHAR(50)`	`UNIQUE`, `NOT NULL`	The user's chosen name for logging in.
`password_hash`	`VARCHAR(255)`	`NOT NULL`	The user's password. (Note: The name `_hash` is a misnomer in the current code; it stores the password in plain text. This is a security risk and should be updated to store a real hash).

- `DESCRIBE users;` Query Result:

```
mysql> desc users;
+---------------+--------------+------+-----+---------+----------------+
| Field         | Type         | Null | Key | Default | Extra          |
+---------------+--------------+------+-----+---------+----------------+
| id            | int          | NO   | PRI | NULL    | auto_increment |
| username      | varchar(50)  | NO   | UNI | NULL    |                |
| password_hash | varchar(255) | NO   |     | NULL    |                |
+---------------+--------------+------+-----+---------+----------------+
3 rows in set (0.03 sec)
```

3.3. Table 2: `monthly_budget`

- **Purpose:** Stores the budget plans created by users. Each row represents one monthly budget for one user.
- **SQL Definition:**

```
CREATE TABLE IF NOT EXISTS monthly_budget (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    month VARCHAR(7) NOT NULL,
    budget FLOAT NOT NULL,
    priority1 VARCHAR(100),
    priority1_percent INT,
    priority2 VARCHAR(100),
    priority2_percent INT,
    priority3 VARCHAR(100),
    priority3_percent INT,
    miscellaneous_percent INT
) ENGINE=InnoDB
```

- **Column Breakdown:**

Column	Type	Purpose
`id`	`INT`	`AUTO_INCREMENT`, `PRIMARY KEY`
`user_id`	`INT`	Links this budget to a user in the `users` table.
`month`	`VARCHAR(7)`	The month for this budget (e.g., "2025-11").
`budget`	`FLOAT`	The total budget amount in ₹.
`priority1`	`VARCHAR(100)`	The name of the user's #1 priority (e.g., "Rent").
`priority1_percent`	`INT`	The percentage allocated to priority 1.
`priority2`	`VARCHAR(100)`	The name of the user's #2 priority.
`priority2_percent`	`INT`	The percentage allocated to priority 2.
`priority3`	`VARCHAR(100)`	The name of the user's #3 priority.
`priority3_percent`	`INT`	The percentage allocated to priority 3.
`miscellaneous_percent`	`INT`	The remaining percentage for misc. spending.

- `DESCRIBE monthly_budget;` Query Result:

```
mysql> desc monthly_budget;
+----------------------+-------------+------+-----+---------+----------------+
| Field                | Type        | Null | Key | Default | Extra          |
+----------------------+-------------+------+-----+---------+----------------+
| id                   | int         | NO   | PRI | NULL    | auto_increment |
| user_id              | int         | NO   |     | NULL    |                |
| month                | varchar(7)  | NO   |     | NULL    |                |
| budget               | float       | NO   |     | NULL    |                |
| priority1            | varchar(100)| YES  |     | NULL    |                |
| priority1_percent    | int         | YES  |     | NULL    |                |
| priority2            | varchar(100)| YES  |     | NULL    |                |
| priority2_percent    | int         | YES  |     | NULL    |                |
| priority3            | varchar(100)| YES  |     | NULL    |                |
| priority3_percent    | int         | YES  |     | NULL    |                |
| miscellaneous_percent| int         | YES  |     | NULL    |                |
+----------------------+-------------+------+-----+---------+----------------+
11 rows in set (0.01 sec)
```

PageBreak

3.4. Table 3: `bill_splits`

- **Purpose:** Keeps a historical record of all bills split using the tool.
- **SQL Definition:**

```
CREATE TABLE IF NOT EXISTS bill_splits (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    bill_amount FLOAT NOT NULL,
    num_people INT NOT NULL,
    per_person_share FLOAT NOT NULL,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB
```

- **Column Breakdown:**

Column	Type	Purpose
`id`	`INT`	`AUTO_INCREMENT`, `PRIMARY KEY`
`user_id`	`INT`	Links this split to the user who performed it.
`bill_amount`	`FLOAT`	The total amount of the bill.
`num_people`	`INT`	The number of people the bill was split amongst.
`per_person_share`	`FLOAT`	The calculated amount each person owes.
`timestamp`	`DATETIME`	The date and time the split was calculated.

- `DESCRIBE bill_splits;` **Query Result:**

```
mysql> desc bill_splits;
+------------------+----------+------+-----+-------------------+-------------------+
| Field            | Type     | Null | Key | Default           | Extra             |
+------------------+----------+------+-----+-------------------+-------------------+
| id               | int      | NO   | PRI | NULL              | auto_increment    |
| user_id          | int      | NO   |     | NULL              |                   |
| bill_amount      | float    | NO   |     | NULL              |                   |
| num_people       | int      | NO   |     | NULL              |                   |
| per_person_share | float    | NO   |     | NULL              |                   |
| timestamp        | datetime | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+------------------+----------+------+-----+-------------------+-------------------+
6 rows in set (0.01 sec)
```

PageBreak

3.5. Table 4: `tax_calculations`

- **Purpose:** Stores a history of all tax calculations performed by the user.
- **SQL Definition:**

```
CREATE TABLE IF NOT EXISTS tax_calculations (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    income FLOAT NOT NULL,
    tax FLOAT NOT NULL,
    effective_rate FLOAT NOT NULL,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB
```

- **Column Breakdown:**

Column	Type	Purpose
`id`	`INT`	`AUTO_INCREMENT`, `PRIMARY KEY`
`user_id`	`INT`	Links this calculation to a user.
`income`	`FLOAT`	The annual income input by the user.
`tax`	`FLOAT`	The total tax calculated as due.
`effective_rate`	`FLOAT`	The tax as a percentage of the total income.
`timestamp`	`DATETIME`	The date and time the calculation was made.

- `DESCRIBE tax_calculations;` **Query Result:**

```
mysql> desc tax_calculations;
+----------------+----------+------+-----+-------------------+-------------------+
| Field          | Type     | Null | Key | Default           | Extra             |
+----------------+----------+------+-----+-------------------+-------------------+
| id             | int      | NO   | PRI | NULL              | auto_increment    |
| user_id        | int      | NO   |     | NULL              |                   |
| income         | float    | NO   |     | NULL              |                   |
| tax            | float    | NO   |     | NULL              |                   |
| effective_rate | float    | NO   |     | NULL              |                   |
| timestamp      | datetime | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+----------------+----------+------+-----+-------------------+-------------------+
6 rows in set (0.00 sec)
```

PageBreak

3.6. Table 5: `compound_interest_records`

- **Purpose:** Stores the inputs and results of compound interest projections.
- **SQL Definition:**

```
CREATE TABLE IF NOT EXISTS compound_interest_records (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    principal FLOAT NOT NULL,
    monthly_contrib FLOAT NOT NULL,
    years FLOAT NOT NULL,
```

```
    annual_rate FLOAT NOT NULL,
    variance FLOAT NOT NULL,
    frequency ENUM('annual','quarterly','monthly'),
    final_amount FLOAT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB
```

- **Column Breakdown:**

Column	Type	Purpose
`id`	`INT`	`AUTO_INCREMENT`, `PRIMARY KEY`
`user_id`	`INT`	Links this projection to a user.
`principal`	`FLOAT`	The initial investment amount.
`monthly_contrib`	`FLOAT`	The extra amount added each month.
`years`	`FLOAT`	The duration of the investment.
`annual_rate`	`FLOAT`	The expected annual interest rate.
`variance`	`FLOAT`	The variance % used for high/low projections.
`frequency`	`ENUM(...)`	How often the interest is compounded.
`final_amount`	`FLOAT`	The final projected balance (at the *expected* rate).
`timestamp`	`DATETIME`	The date and time of the projection.

- `DESCRIBE compound_interest_records;` **Query Result:**

```
mysql> desc compound_interest_records;
+-----------------+-------------------------------------+------+-----+-------------------+-------------------+
| Field           | Type                                | Null | Key | Default           | Extra             |
+-----------------+-------------------------------------+------+-----+-------------------+-------------------+
| id              | int                                 | NO   | PRI | NULL              | auto_increment    |
| user_id         | int                                 | NO   |     | NULL              |                   |
| principal       | float                               | NO   |     | NULL              |                   |
| monthly_contrib | float                               | NO   |     | NULL              |                   |
| years           | float                               | NO   |     | NULL              |                   |
| annual_rate     | float                               | NO   |     | NULL              |                   |
| variance        | float                               | NO   |     | NULL              |                   |
| frequency       | enum('annual','quarterly','monthly') | YES  |     | NULL              |                   |
| final_amount    | float                               | YES  |     | NULL              |                   |
| timestamp       | datetime                            | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----------------+-------------------------------------+------+-----+-------------------+-------------------+
10 rows in set (0.01 sec)
```

PageBreak

3.7. Table 6: `currency_conversions`

- **Purpose:** Logs all currency conversions, whether they used live or fixed rates.
- **SQL Definition:**

```
CREATE TABLE IF NOT EXISTS currency_conversions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    from_currency VARCHAR(3) NOT NULL,
    to_currency VARCHAR(3) NOT NULL,
    amount FLOAT NOT NULL,
    converted_amount FLOAT NOT NULL,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB
```

- **Column Breakdown:**

Column	Type	Purpose
`id`	`INT`	`AUTO_INCREMENT`, `PRIMARY KEY`
`user_id`	`INT`	Links this conversion to a user.
`from_currency`	`VARCHAR(3)`	The 3-letter code of the starting currency (e.g., "USD").
`to_currency`	`VARCHAR(3)`	The 3-letter code of the target currency (e.g., "INR").
`amount`	`FLOAT`	The amount of `from_currency` that was converted.
`converted_amount`	`FLOAT`	The resulting amount in `to_currency`.
`timestamp`	`DATETIME`	The date and time of the conversion.

- `DESCRIBE currency_conversions;` **Query Result:**

```
mysql> DESCRIBE currency_conversions;
+------------------+------------+------+-----+-------------------+-------------------+
| Field            | Type       | Null | Key | Default           | Extra             |
+------------------+------------+------+-----+-------------------+-------------------+
| id               | int        | NO   | PRI | NULL              | auto_increment    |
| user_id          | int        | NO   |     | NULL              |                   |
| from_currency    | varchar(3) | NO   |     | NULL              |                   |
| to_currency      | varchar(3) | NO   |     | NULL              |                   |
| amount           | float      | NO   |     | NULL              |                   |
| converted_amount | float      | NO   |     | NULL              |                   |
| timestamp        | datetime   | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+------------------+------------+------+-----+-------------------+-------------------+
7 rows in set (0.01 sec)
```

PageBreak

3.8. Table 7: `savings_records`

- **Purpose:** Stores the inputs and results of savings plan projections.
- **SQL Definition:**

```
CREATE TABLE IF NOT EXISTS savings_records (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    monthly_saving FLOAT NOT NULL,
    years FLOAT NOT NULL,
    annual_return FLOAT NOT NULL,
    final_balance FLOAT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB
```

- **Column Breakdown:**

Column	Type	Purpose
`id`	`INT`	`AUTO_INCREMENT`, `PRIMARY KEY`
`user_id`	`INT`	Links this projection to a user.
`monthly_saving`	`FLOAT`	The amount saved each month.
`years`	`FLOAT`	The duration of the savings plan.
`annual_return`	`FLOAT`	The expected annual return rate (e.g., from interest).
`final_balance`	`FLOAT`	The total projected balance after the full duration.

Column	Type	Purpose
`timestamp`	`DATETIME`	The date and time of the projection.

- `DESCRIBE savings_records;` **Query Result:**

```
mysql> DESCRIBE savings_records;
+----------------+----------+------+-----+-------------------+-------------------+
| Field          | Type     | Null | Key | Default           | Extra             |
+----------------+----------+------+-----+-------------------+-------------------+
| id             | int      | NO   | PRI | NULL              | auto_increment    |
| user_id        | int      | NO   |     | NULL              |                   |
| monthly_saving | float    | NO   |     | NULL              |                   |
| years          | float    | NO   |     | NULL              |                   |
| annual_return  | float    | NO   |     | NULL              |                   |
| final_balance  | float    | YES  |     | NULL              |                   |
| timestamp      | datetime | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+----------------+----------+------+-----+-------------------+-------------------+
7 rows in set (0.00 sec)
```

PageBreak

3.9. Table 8: `entries`

- **Purpose:** This is the core financial ledger. It stores every individual income and expense transaction for all users.
- **SQL Definition:**

```
CREATE TABLE IF NOT EXISTS entries (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    type ENUM('income','expense'),
    amount FLOAT NOT NULL,
    tag VARCHAR(100) NOT NULL,
    entry_time DATETIME NOT NULL
) ENGINE=InnoDB
```

- **Column Breakdown:**

Column	Type	Purpose
`id`	`INT`	`AUTO_INCREMENT`, `PRIMARY KEY`
`user_id`	`INT`	Links this transaction to a user.
`type`	`ENUM('income','expense')`	Specifies if the transaction is money coming in or out.
`amount`	`FLOAT`	The value of the transaction (always positive).
`tag`	`VARCHAR(100)`	A user-defined category (e.g., "salary", "food", "rent").
`entry_time`	`DATETIME`	The exact date and time the entry was recorded.

- `DESCRIBE entries;` **Query Result:**

```
mysql> DESCRIBE entries;
+------------+--------------------------+------+-----+---------+----------------+
| Field      | Type                     | Null | Key | Default | Extra          |
+------------+--------------------------+------+-----+---------+----------------+
| id         | int                      | NO   | PRI | NULL    | auto_increment |
| user_id    | int                      | NO   |     | NULL    |                |
| type       | enum('income','expense') | YES  |     | NULL    |                |
| amount     | float                    | NO   |     | NULL    |                |
| tag        | varchar(100)             | NO   |     | NULL    |                |
| entry_time | datetime                 | NO   |     | NULL    |                |
+------------+--------------------------+------+-----+---------+----------------+
6 rows in set (0.01 sec)
```

PageBreak

Chapter 4: Full Code Analysis

This chapter provides a detailed, function-by-function analysis of the `Finance_Bot_FMM.py` source code. Functions are grouped by their overall purpose.

4.1. Imports and Global Configuration

This section defines the libraries and global variables used throughout the script.

```
import mysql.connector
import datetime
import matplotlib.pyplot as plt
from forex_python.converter import CurrencyRates

DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "javamx",
    "database": "college_finance_db"
}

CURRENCIES = {
    "INR": "Indian Rupee",
    "USD": "US Dollar",
    "EUR": "Euro",
    "GBP": "British Pound",
    "JPY": "Japanese Yen",
    "AUD": "Australian Dollar",
    "CAD": "Canadian Dollar",
    "CHF": "Swiss Franc",
    "CNY": "Chinese Yuan",
    "NZD": "New Zealand Dollar",
}

conversion_rates = {
    "USD": {"INR_to_currency": 0.01133, "currency_to_INR": 88.28},
    "EUR": {"INR_to_currency": 0.00965, "currency_to_INR": 103.60},
    "GBP": {"INR_to_currency": 0.00835, "currency_to_INR": 119.66},
    "JPY": {"INR_to_currency": 1.67, "currency_to_INR": 0.598},
    "AUD": {"INR_to_currency": 0.01722, "currency_to_INR": 58.08},
    "CAD": {"INR_to_currency": 0.0141, "currency_to_INR": 70.92},
    "CHF": {"INR_to_currency": 0.0108, "currency_to_INR": 92.60},
```

```
    "CNY": {"INR_to_currency": 0.0807, "currency_to_INR": 12.29},
    "NZD": {"INR_to_currency": 0.0190, "currency_to_INR": 52.59},
}
```

- **Explanation:**
  - `mysql.connector` : Imports the MySQL driver.
  - `datetime` : Imports the module for
  - `matplotlib.pyplot as plt` : Imports the plotting library, aliased as `plt` .
  - `forex_python.converter` : Imports the `CurrencyRates` class for live conversion.
  - `DB_CONFIG` : A global dictionary holding the database credentials. This is a critical configuration point.
  - `CURRENCIES` : A dictionary mapping 3-letter currency codes to their full names for display purposes.
  - `conversion_rates` : A dictionary of *fixed, offline* exchange rates. This is used as a fallback in the `currency_converter` function if the user chooses not to use the live API.

4.2. Helper & Utility Functions

These functions provide support services used by other parts of the application.

4.2.1. `convert_inr_to_currency()`

- **Purpose:** Converts a given amount *from* INR *to* a foreign currency, using the fixed, offline `conversion_rates` dictionary.
- **Parameters:**
  - `amount_inr` (float): The amount in Indian Rupees.
  - `currency_code` (str): The 3-letter code of the target currency (e.g., "USD").
- **Returns:** (float) The equivalent amount in the target currency.
- **Logic Explanation:** This function checks if the requested `currency_code` exists in the offline `conversion_rates` dictionary. If not, it raises an error. If it exists, it retrieves the "INR_to_currency" rate and returns the calculated converted amount.

4.2.2. `convert_currency_to_inr()`

- **Purpose:** Converts a given amount *from* a foreign currency *to* INR, using the fixed, offline `conversion_rates` dictionary.
- **Parameters:**
  - `amount_currency` (float): The amount in the foreign currency.
  - `currency_code` (str): The 3-letter code of the starting currency (e.g., "USD").
- **Returns:** (float) The equivalent amount in Indian Rupees.
- **Logic Explanation:** This function works identically to its counterpart. It checks the `conversion_rates` dictionary for the `currency_code` , retrieves the "currency_to_INR" rate, and returns the calculated amount in Rupees.

4.2.3. `print_banner()`

- **Purpose:** A simple formatting utility to print a clean, centered title banner to the console. This improves the user interface's readability.
- **Parameters:**
  - `text` (str): The text to be displayed in the banner.

- **Returns:** None.
- **Logic Explanation:** A simple formatting utility that prints a 60-character line of equals signs, followed by the provided `text` centered within 60 characters, and another line of equals signs. This creates a clean title banner in the console.

4.3. Database Connection & Initialization

These functions manage the database connection and initial setup.

4.3.1. `db_connect()`

- **Purpose:** This is the primary function for connecting to the MySQL database. It uses the `DB_CONFIG` global variable.
- **Parameters:** None.
- **Returns:** A tuple `(cnx, cursor)` where `cnx` is the database connection object and `cursor` is the object used to execute queries.
- **Logic Explanation:** The function attempts to connect to the MySQL database using the global `DB_CONFIG`. If successful, it creates and returns the connection (`cnx`) and cursor (`cursor`) objects. If the connection fails (e.g., wrong password, server offline), it prints a fatal error message and terminates the entire script with `exit(1)`, as the application cannot run without a database.

4.3.2. `create_all_tables()`

- **Purpose:** This crucial function is called once when the application starts. It ensures the database and all 8 tables exist, creating them if they don't.
- **Parameters:** None.
- **Returns:** None.
- **Logic Explanation:** This function is called once at startup. It first connects to the MySQL *server* (without specifying a database) to execute the `CREATE DATABASE IF NOT EXISTS` command. It then selects that database and iterates through a dictionary named `table_ddls` which contains the SQL definitions for all 8 tables. It executes each `CREATE TABLE IF NOT EXISTS` command, ensuring the full database schema is ready for use. It will exit the script if any database or table creation fails.

4.4. User Authentication Functions

These functions handle user registration and login.

4.4.1. `register_user()`

- **Purpose:** To guide a new user through the process of creating a secure account.
- **Logic Explanation:** This function guides a new user through registration. It opens a database connection and enters a loop asking for a username. It validates that the username is at least 3 characters and queries the database to ensure the username is not already taken. Once a valid username is provided, it enters a second loop for password validation, checking that the password is at least 6 characters and that the confirmation matches. Finally, it inserts the new `username` and `password` into the `users` table, commits the change, and instructs the user to log in.

4.4.2. `login_user()`

- **Purpose:** To verify a returning user's credentials and grant them access.

- **Returns:** The user's `id` (int) on successful login, or `None` on failure.
- **Logic Explanation:** This function handles user login. It gets a username and password from the user. It then queries the `users` table for a matching `username`, selecting the user's `id` and stored `password_hash`. If no user is found (`row is None`), it returns `None`. If a user is found, it compares the input `password` with the `stored_password` from the database. If they match, it returns the `user_id`. If they don't match, it prints an error and returns `None`.

PageBreak

4.5. Core Financial Feature Functions

This section details all 8 financial tools available to a logged-in user.

4.5.1. `get_user_financial_summary()`

- **Purpose:** A helper function to quickly calculate a user's total income, total expense, and current balance.
- **Parameters:**
  - `user_id` (int): The ID of the currently logged-in user.
- **Returns:** A tuple `(total_income, total_expense, balance)`.
- **Logic Explanation:** A helper function that runs two separate `SELECT SUM(amount)` queries on the `entries` table for the given `user_id`. The first query sums all `'income'` types, and the second sums all `'expense'` types. It uses `or 0` to handle cases where there are no entries (sum is `None`). It then returns the `total_income`, `total_expense`, and the calculated `balance` (income - expense).

4.5.2. `record_entry()`

- **Purpose:** This is the main ledger function. It allows users to add a new income or expense transaction to their account.
- **Logic Explanation:** This is the main ledger function. It first asks the user for the entry type ('i' or 'e') and validates the input. It then gets a valid positive `amount`. If the type is 'income', it forces the user to select a tag from a pre-defined list. If it's an 'expense', the user can enter any tag. It gets the current `datetime`, then inserts the new transaction into the `entries` table. After committing, it calls `get_user_financial_summary()` to fetch the new totals and prints a formatted confirmation message to the user.

4.5.3. `monthly_budget_planner()`

- **Purpose:** To guide the user in creating a monthly budget and save it to the database.
- **Logic Explanation:** This function guides the user in creating a budget. It gets a total budget amount, three priority names, and the percentage for each priority. It validates that the percentages are valid and that the total does not exceed 100. It then calculates the remaining 'miscellaneous' percentage. It gets the current month (e.g., '2025-11'), inserts the full budget plan into the `monthly_budget` table, and commits. Finally, it prints a formatted breakdown of the budget in both amounts and percentages.

4.5.4. `bill_splitter()`

- **Purpose:** A simple utility to divide a bill and log the event.
- **Logic Explanation:** A simple utility that gets a total bill amount and a number of people. It validates that the number of people is a positive integer. It then calculates the `share` (total / people), inserts the record into the `bill_splits` table for history, commits the change, and prints the per-person share formatted to two decimal places.

4.5.5. `tax_calculator()`

- **Purpose:** To calculate income tax based on a progressive, hard-coded tax slab system.
- **Logic Explanation:** This function calculates income tax based on a hard-coded, progressive slab system. It defines the slabs and rates in a list. It then loops through the slabs, calculating the tax for the portion of income in each bracket and subtracting that portion from the 'remaining' income. After the loop, it calculates the `effective_rate` (total tax / income), inserts the record into the `tax_calculations` table, commits, and prints the total tax due and the effective rate.

4.5.6. `compound_interest()`

- **Purpose:** A complex projection tool that calculates and *graphs* the growth of an investment under three scenarios (low, expected, high).
- **Logic Explanation:** A complex projection tool. It gathers 6 inputs from the user (principal, monthly contribution, years, rate, variance, and frequency). It creates three rate scenarios (low, mid, high) using the `variance`. It then loops through each rate scenario, simulating month-by-month growth and storing the balance. The core logic correctly handles different compounding frequencies. After all scenarios are calculated, it uses `matplotlib` to plot all three on a graph and calls `plt.show()` to display it. It then saves the *expected* (mid-rate) projection to the `compound_interest_records` table and prints a text summary of all three scenarios.

4.5.7. `savings_calculator()`

- **Purpose:** A simpler projection tool that calculates and graphs the growth of a regular monthly savings plan.
- **Logic Explanation:** A simpler projection tool. It gets the monthly saving amount, duration in years, and expected annual return. It calculates the monthly interest rate and loops for the total number of months. In each loop, it calculates the new balance by adding interest to the previous balance, then adding the new month's saving. It plots the resulting list of balances using `matplotlib` and calls `plt.show()`. Finally, it saves the projection to the `savings_records` table and prints the final balance.

4.5.8. `currency_converter()`

- **Purpose:** To convert between currencies, offering a choice between live (API) or fixed (offline) rates.
- **Logic Explanation:** This function converts between currencies. It first initializes the `CurrencyRates` object. In a loop, it gets the amount, 'from' currency, and 'to' currency. It then asks the user if they want to use 'fixed offline rates'. If 'yes', it uses the `convert_inr_to_currency` and `convert_currency_to_inr` helper functions (handling `XXX -> YYY` conversions via INR). If 'no', it uses the `c.convert()` method to get a live API rate. In both cases, it saves the transaction to the `currency_conversions` table, commits, and asks the user if they want to perform another conversion.

PageBreak

4.6. Main Application Flow

These functions control the overall execution and user interface of the application.

4.6.1. `main_menu()`

- **Purpose:** A simple function that prints the 8-option menu for a logged-in user and returns their choice.
- **Returns:** (str) The user's typed choice (e.g., "1", "5").
- **Logic Explanation:** A very simple function that prints the 8-option main menu for a logged-in user and returns the string of the user's choice (e.g., '1', '8').

4.6.2. `main()`

- **Purpose:** This is the main "controller" function for the entire application. It orchestrates the program flow, from initialization to login to the main feature loop.
- **Logic Explanation:** This is the main controller function. It first calls `create_all_tables()` to ensure the database is ready. It then enters the 'login/registration loop', which only breaks after a successful login (when `login_user()` returns a `user_id`).
- After a successful login. In this loop, it calls `main_menu()`, gets the user's choice, and uses a large `if/elif` block to call the corresponding function. If the choice is '8' (Logout), it breaks this loop, and the program ends.

4.6.3. `if __name__ == "__main__":`

- **Purpose:** A standard Python idiom that checks if the script is being run directly by the user (as opposed to being imported as a module into another script).
- **Logic Explanation:** This standard Python construct checks if the script is being executed directly (not imported). If it is, it calls the `main()` function, which starts the entire application.

PageBreak

Chapter 5: Execution & Usage Guide

5.1. Running the Application

To run the application, ensure you have completed all steps in **Chapter 2 (System & Setup Requirements)**.

1. Open your terminal or command prompt.
2. Navigate to the directory containing the `Finance_Bot_FMM.py` file.
3. Type the following command and press Enter:

```
python Finance_Bot_FMM.py
```

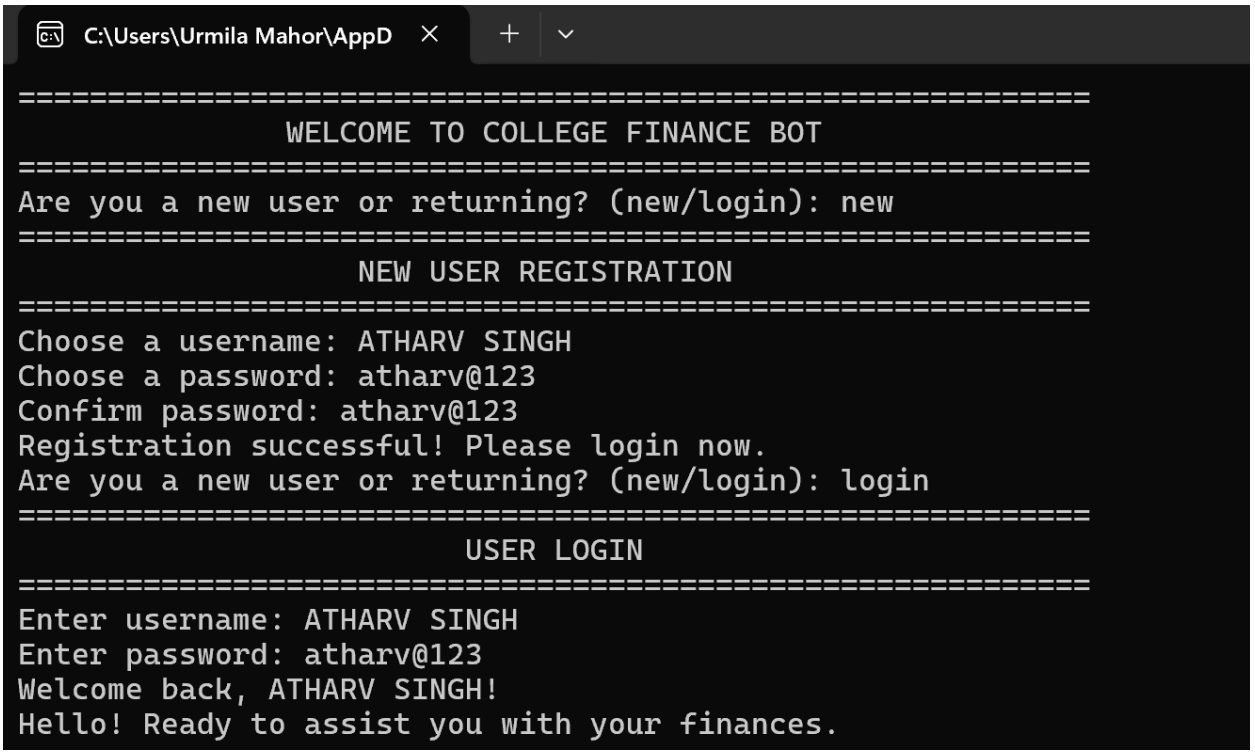
This will start the application, which will first connect to the database (creating it if needed) and then present you with the initial login/registration prompt.

5.2. Execution Walkthrough (Screenshot Placeholders)

This section provides a visual walkthrough of the application's key features. Please insert your own execution screenshots in the spaces provided.

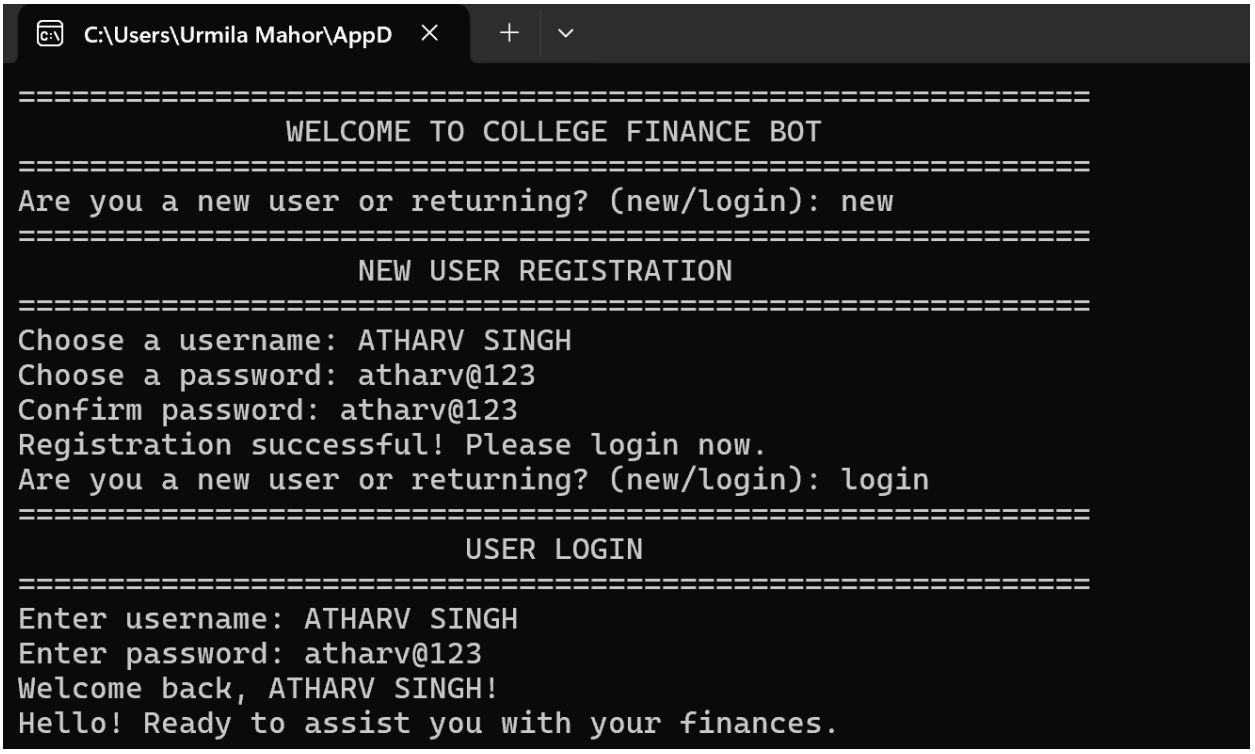
5.2.1. Execution 1: Registration

- **User Flow:**
  1. Run the script.
  2. At the prompt `Are you a new user or returning? (new/login):`, type `new`.
  3. Follow the prompts to enter a new username, a password, and confirm the password.
  4. The application will display "Registration successful! Please login now."
- **Screenshot Placeholder:**



5.2.2. Execution 2: Login & Main Menu

- **User Flow:**
  1. Run the script.
  2. At the prompt `Are you a new user or returning? (new/login):`, type `login`.
  3. Enter the username and password you just registered.
  4. The application will display the main 8-option "USER MENU".
- **Screenshot Placeholder:**



5.2.3. Execution 3: Recording an Expense

- **User Flow:**
  1. From the main menu, select option `7`.
  2. Enter `e` for expense.
  3. Enter an amount (e.g., `500`).
  4. Enter a tag (e.g., `Groceries`).
  5. The application will display a summary of the recorded expense and your new balance.
- **Screenshot Placeholder:**

```
============================================================
                RECORD INCOME/EXPENSE ENTRY
============================================================
Enter 'i' for income or 'e' for expense: i
Enter amount (₹): 500000
Allowed income tags: salary, bonus, interest, investment, gift, other
Enter a tag for income: salary

============================================================
🎉 Income recorded: ₹500000.00 (Cr)
Total Income: ₹500000.00 (Cr)
============================================================

Hello! Your income entry is safely logged.
```

5.2.4. Execution 4: Planning a Monthly Budget

- **User Flow:**
  1. From the main menu, select option `1`.
  2. Enter a total budget (e.g., `50000`).
  3. Enter three priority names (e.g., `Rent`, `Food`, `Transport`).
  4. Enter percentages for each (e.g., `50`, `20`, `10`).
  5. The application will display a full breakdown of the budget, including the calculated "Miscellaneous" amount.
- **Screenshot Placeholder:**

```
============================================================
                  MONTHLY BUDGET PLANNER
============================================================
Enter your total monthly budget (₹): 50000
Enter priority #1 expense name: Food
Enter priority #2 expense name: Laundary
Enter priority #3 expense name: Printing
Enter % allocation for Food: 20
Enter % allocation for Laundary: 30
Enter % allocation for Printing: 10

Budget Breakdown:
Category                          Amount   Percent
--------------------------------------------------
Food                             10000.0        20%
Laundary                         15000.0        30%
Printing                          5000.0        10%
Miscellaneous                    20000.0        40%
==================================================
Great job on planning your budget for the month!
```

5.2.5. Execution 5: Calculating Tax

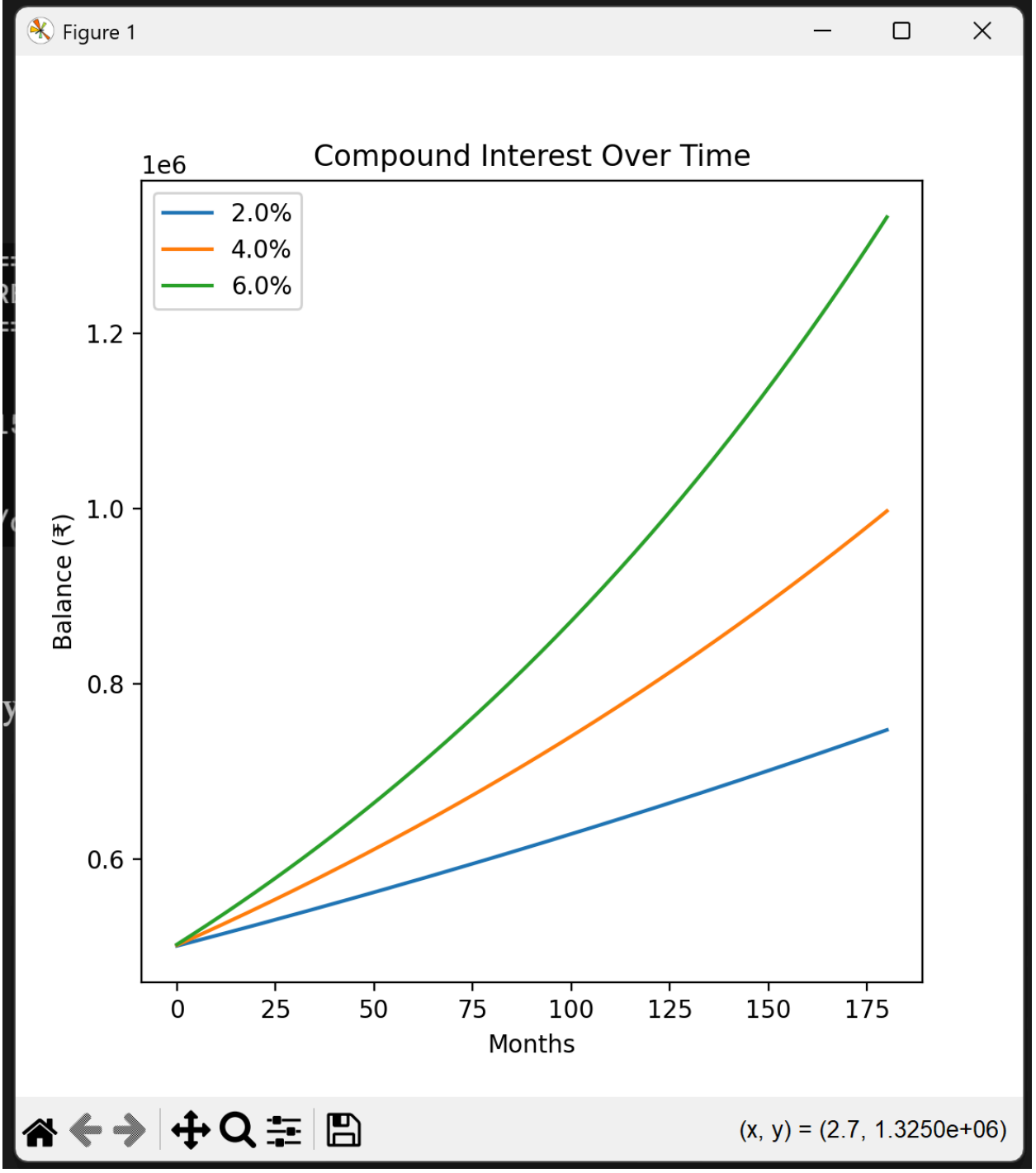
- **User Flow:**
  1. From the main menu, select option `3`.
  2. Enter an annual income (e.g., `750000`).
  3. The application will display the calculated "Tax due" and "Effective tax rate".
- **Screenshot Placeholder:**

```
Select an option (1-8): 3
============================================================
                        TAX CALCULATOR
============================================================
Enter annual income (₹): 1200000
Tax due: ₹172500.00
Effective tax rate: 14.37%
Tax calculation done. Keep your records updated!
```

5.2.6. Execution 6: Projecting Compound Interest (Graph)

- **User Flow:**
  1. From the main menu, select option `4`.
  2. Enter a principal (e.g., `10000`).
  3. Enter a monthly contribution (e.g., `500`).
  4. Enter duration (e.g., `10` years).
  5. Enter annual rate (e.g., `8` %).
  6. Enter variance (e.g., `2` %).
  7. Enter frequency (e.g., `monthly`).
  8. **A new window will pop up displaying the line graph.** The terminal will show the text summary.
- **Screenshot Placeholder:**

```
============================================================
                COMPOUND INTEREST CALCULATOR
============================================================
Principal amount (₹): 500000
Monthly contribution (₹): 340
Investment duration (years): 15
Annual interest rate (%): 4
Variance (%): 2
Compounding frequency (annual/quarterly/monthly): monthly
```



5.2.7. Execution 7: Converting Currency (Live Rates)

- **User Flow:**
  1. From the main menu, select option `5`.
  2. Enter an amount (e.g., `100`).
  3. Enter From currency (e.g., `USD`).

4. Enter To currency (e.g., `INR`).
5. When asked to use fixed rates, type `no`.
6. The application will display the live conversion result (e.g., `100.00 USD = 8345.50 INR`).

- **Screenshot Placeholder:**

```
============================================================
                    CURRENCY CONVERTER
============================================================
Supported currencies:
INR - Indian Rupee
USD - US Dollar
EUR - Euro
GBP - British Pound
JPY - Japanese Yen
AUD - Australian Dollar
CAD - Canadian Dollar
CHF - Swiss Franc
CNY - Chinese Yuan
NZD - New Zealand Dollar
Enter amount: 80000
From currency code: JPY
To currency code: USD
Use fixed offline rates instead of live rates? (yes/no): yes
80000.00 JPY = 542.0272 USD
Currency conversion successful!
Convert another? (yes/no): no
```

5.2.8. Execution 8: Logout

- **User Flow:**
  1. From the main menu, select option `8`.
  2. The application will display "Goodbye! Have a great day!" and the script will terminate.

PageBreak

Chapter 6: Conclusion

6.1. Document Summary

This manual has provided a complete and comprehensive analysis of the **College Finance Bot** (`Finance_Bot_FMM.py`) application. It has covered the project's intended purpose, all software and setup requirements, a detailed breakdown of the 8-table MySQL database schema, and a functional analysis of the entire Python source code.

The application stands as a robust, feature-rich financial tool. The code is logically structured, with clear separation of concerns between database initialization, user authentication, and core financial utilities. The use of a persistent database makes it a powerful and reusable tool for a personal user.

This document serves as a complete technical record of the project in its current state.

6.1. Document Summary

- Python 3 Ocial Documentation. (2025). The Python Standard Library. Retrieved from docs.python.org MySQL Documentation. (2025).
- MySQL 8.0 Reference Manual. Retrieved from https://www.google.com/search? q=dev.mysql.com/doc Hunter, J. D. (2007).

- Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95. mysql-connector-python Developer Guide. (2025).
- Retrieved from https://www.google.com/search? q=dev.mysql.com/doc/connector-python/en/ forex-python Library Documentation. (2025). Retrieved from pypi.org/project/forex-python/